

```
#!/usr/bin/env python3
```

```
"""
```

```
06_provenance_gate.py - Stray-paste /  
fabrication / duplication gate.
```

Run this from the repo root (or drop it in
bootstrap/ so 00_run_all.sh
picks it up automatically via its [0-9][0-
9]*.py glob).

Fails (exit 1) if it finds:

1. AI-session transcript artifacts
leaking into data/docs files
(e.g. "Model: <name>", "--- User ---",
"<citation src=...>").
2. The same county -> rating-area mapping
hardcoded in more than one
.py file (SSOT violation - crosswalk
should live in exactly one place).
3. Any citation string listed in this
repo's own
data/statutes_2026.json
"excluded_citations" resurfacing anywhere
else in the repo as if it were live.

This does NOT verify facts against primary
sources. It only catches the
specific failure patterns that have already
recurred in this project's
history. A clean run means "nothing

```

obviously contaminated the repo,"
not "every number is confirmed."
"""

import json
import re
import sys
from pathlib import Path

ROOT =
Path(__file__).resolve().parent.parent
FAILURES = []

AI_ARTIFACT_PATTERNS = [
    r"^\\s*Model:\\s*\\S+",
    r"^\\s*-{2,}\\s*User\\s*-{2,}",
    r"^\\s*-{2,}\\s*Assistant\\s*-{2,}",
    r"<citation\\s+src=",
    r"\\bas an AI (language )?model\\b",
]

SKIP_DIRS = {".git", "node_modules",
"__pycache__", ".venv"}
SCAN_EXTS = {".txt", ".md", ".json"}

def iter_files(exts):
    for path in ROOT.rglob("*"):
        if not path.is_file():
            continue
        if any(part in SKIP_DIRS for part
in path.parts):
            continue
        if path.suffix.lower() in exts:
            yield path

```

```

def read(path):
    try:
        return
        path.read_text(encoding="utf-8",
            errors="ignore")
    except Exception:
        return None

def scan_ai_artifacts():
    for path in iter_files(SCAN_EXTS):
        text = read(path)
        if text is None:
            continue
        for pat in AI_ARTIFACT_PATTERNS:
            if re.search(pat, text,
                re.MULTILINE | re.IGNORECASE):
                FAILURES.append(f"[AI-
ARTIFACT] {path.relative_to(ROOT)}: matches
/{pat}/")

def scan_duplicate_crosswalks():
    """Flag any two .py files that both
    hardcode a near-complete
    county->rating-area map (heuristic: 40+
    'County': digit' pairs)."""
    county_maps = {}
    for path in iter_files({".py"}):
        text = read(path)
        if text is None:
            continue

```

```

        counties = set(re.findall(r'"([A-Z]
[a-zA-Z]+)":\s*[1-9]\b', text))
        if len(counties) >= 40:
            county_maps[path] = counties

    paths = list(county_maps.keys())
    for i in range(len(paths)):
        for j in range(i + 1, len(paths)):
            overlap = county_maps[paths[i]]
& county_maps[paths[j]]
            if len(overlap) >= 40:
                FAILURES.append(
                    f"[DUPLICATE-SSOT]
{paths[i].relative_to(ROOT)} and "
                    f"
{paths[j].relative_to(ROOT)} both hardcode
the "
                    f"county->rating-area
map ({len(overlap)} counties overlap). "
                    f"Pick one as the
source of truth; have the other import it."
                )

def scan_excluded_citations_resurfacing():
    statutes_file = ROOT / "data" /
"statutes_2026.json"
    if not statutes_file.exists():
        # not fatal - just means this check
can't run yet
        return
    try:
        data =
json.loads(statutes_file.read_text(encoding

```

```

="utf-8"))
    except Exception as e:
        FAILURES.append(f"[BAD-JSON]
{statutes_file.relative_to(ROOT)}: {e}")
        return

    excluded = [item["citation"] for item
in data.get("excluded_citations", [])]
    for path in iter_files({".txt", ".md",
".json", ".py"}):
        if path == statutes_file:
            continue
        text = read(path)
        if text is None:
            continue
        for citation in excluded:
            if citation and citation in
text:
                FAILURES.append(
                    f"[EXCLUDED-CITATION-
RESURFACED] {path.relative_to(ROOT)}: "
                    f"contains
'{citation}', which data/statutes_2026.json
"
                    f"already marked
excluded/fabricated."
                )

def main():
    scan_ai_artifacts()
    scan_duplicate_crosswalks()
    scan_excluded_citations_resurfacing()

```

```
    if FAILURES:
        print("=== PROVENANCE GATE: FAILED
===")
        for f in FAILURES:
            print(f)
            print(f"\n{len(FAILURES)} issue(s)
found. Fix before packaging a release.")
            sys.exit(1)

        print("[OK] Provenance gate passed: no
stray AI artifacts, "
              "no duplicate crosswalk SSOT, no
resurfaced excluded citations.")
        sys.exit(0)

if __name__ == "__main__":
    main()
```